PCETS,

PIMPRI CHINCHWAD COLLEGE OF

ENGINEERING AND RESEARCH, RAVET,

PUNE-412101

Department of

COMPUTER ENGINEERING

LABORATORY MANUAL

PROGRAM	COMPUTER ENGINEERING
COURSE	PCC-205-COM (2024)
COURSE NAME	OBJECT ORIENTED PROGRAMMING & COMPUTER GRAPHICS LAB PART II - COMPUTER GRAPHICS
CLASS	SE COMPUTER –SEM I

Savitribai Phule Pune University Second Year of Computer Engineering (2024 Course) PCC-205-COM: OOP and Computer Graphics Laboratory		
Teaching Scheme: **PR: 04 Hours/Week**	**Credit** **02**	**Examination Scheme:** **TW: 50 Marks** **PR: 25 Marks**
Prerequisite Courses, if any: Understanding of Programming and Problem Solving concepts, Basic Mathematics.		
Companion Course, if any: Object oriented Programming, Computer Graphics		

Course Objectives:

The course aims to:

1. To explore & understand the principles of Object-Oriented Programming (OOP).
2. To apply the object-oriented paradigm in program design.
3. To provide object-oriented programming insight using Java
4. To lay a foundation for computer graphics concepts and algorithms

Course Outcomes:

Upon successful completion of this course, students will be able to:

1. **CO1:** To **apply** fundamental programming constructs in Java for implementing an application.
2. **CO2:** To **apply** fundamental object oriented constructs in Java for implementing an application.
3. **CO3:** To **apply** object-oriented features like Inheritance, Polymorphism, Dynamic binding, exception handling, multi-threading in Java for implementing an application.
4. **CO4:** To **implement** basic concepts in computer graphics by applying object oriented features.
5. **CO5:** To **implement** 2D, 3D Transformation, Projections by applying object oriented features.

Guidelines for Instructor's Manual

The instructor's manual/Lab Manual is to be developed as a hands-on resource and reference. The instructor's manual need to include prologue (about University/program/ institute/ department/foreword/ preface), curriculum of course, conduction and Assessment guidelines, topics under consideration-concept, objectives, outcomes, set of typical applications/assignments/guidelines, references.

Guidelines for Student's Laboratory Journal

The laboratory assignments are to be submitted by student in the form of journal. Journal consists of prologue, Certificate, table of contents, and handwritten write-up of each assignment (Title, Objectives, Problem Statement, Outcomes, software and Hardware requirements, Date of Completion, Assessment grade/marks and assessor's sign, Theory Concept in brief, algorithm, flowchart, test cases, Test Data Set(if applicable), mathematical model (if applicable), conclusion/analysis. Program codes with sample output of all performed assignments are to be submitted as softcopy. As a conscious effort and little contribution towards Green IT and environment awareness, attaching printed papers as part of write-ups and program listing to journal may be avoided. Students programs maintained on cloud or college server by Laboratory In-charge is highly encouraged. For reference one or two journals may be maintained with program prints at Laboratory for accreditation purposes.

Guidelines for Laboratory /TW Assessment

Continuous assessment of laboratory work should be done based on overall performance and Laboratory assignments performance of students. Each Laboratory assignment assessment should be assigned grade/marks based on parameters with appropriate weightage. Suggested parameters for overall assessment as well as each Laboratory assignment assessment include timely completion performance, innovation, efficient codes, punctuality and neatness.

Guidelines for Practical Examination

Both internal and external examiners should jointly set problem statements. During practical assessment, the expert evaluator should give the maximum weightage to the satisfactory implementation of the problem statement. The supplementary and relevant questions may be asked at the time of evaluation to test the students for advanced learning, understanding of the fundamentals, effective and efficient implementation. So encouraging efforts, transparent evaluation and fair approach of the evaluator will not create any uncertainty or doubt in the minds of the students. So adhering to these principles will consummate our team efforts to the promising start of the student's academics.

Guidelines for Laboratory Conduction

The instructor is expected to frame the assignments by understanding the prerequisites, technological aspects, utility and recent trends related to the topic. The assignment framing policy needs to address the average

students and inclusive of an element to attract and promote the intelligent students. The instructor may set multiple sets of assignments and distribute them among batches of students.
It is appreciated if the assignments are based on real world problems/applications. Encourage students for appropriate use of Hungarian notation, proper indentation and comments. Use of open source software is to be encouraged. In addition to these, instructors may assign one real life application in the form of a mini-project based on the concepts learned. Instructors may also set one assignment or mini-project that is suitable to respective branch beyond the scope of the syllabus.

Operating System recommended:- 64-bit Open source Linux or its derivative
Programming tools recommended: - Open Source Java Open JDK,
Programming IDE like: BlueJ, Eclipose, NetBeans, JDeveloper.

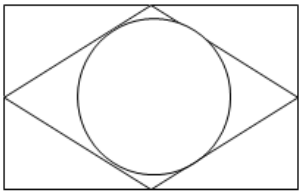
Part-A: 5 Assignments, Part- B: 6 Assignments, Part-C (Mini Project): Mandatory Assignment.

Guidelines for Oral Examination

Oral examination gauge students' knowledge and skills based on the spoken word, typically guided by questions or small tasks. A pair of examiners must design appropriate questions for each learning outcome. They should focus on depth rather than breadth. They should include potential follow up questions and prompts based on different types of answers. Examiners should standardize the number of questions, difficulty of questions, and the time allotted. Questions should be based on the practical assignments performed in the term work and not on the entire syllabus.

Suggested List of Laboratory Experiments/Assignments

(Implementation of each problem statement is mandatory.)

Part I : Object Oriented Programming	
Sr.No.	**Group A - Any THREE (from 1 to 5)**
1.	Implement a robust Java calculator program that captures user input dynamically, processes mathematical operations using conditional logic and looping constructs, and ensures efficient error handling.
2.	Develop a Java program for an E-commerce order processing where some products are initialized through multiple constructors, overloaded constructors, where users can input some product details manually, the system computes total order cost dynamically, applies discount policies based on conditions, and presents a detailed invoice summarizing the purchase.
3.	Develop a Java program that implements a simple hotel room booking system using two dimensional arrays. The system allows users to: View available and booked rooms, Book a room by selecting a floor and room number and exit the system when finished.
4.	Create a Java program demonstrating single inheritance where a subclass extends a superclass and calls its methods.
5.	Implement Multiple Inheritance using interfaces in Java to demonstrate polymorphism.
6.	Develop a Java program for simulation of any real time application with required functionalities. For eg. ATM machine with functionalities like checking account balance, withdrawing, and depositing money. Use try, catch, and finally blocks to handle potential exceptions such as insufficient funds (throwing ArithmeticException) and invalid input (throwing IllegalArgumentException). Ensure that the application continues to run smoothly after handling exceptions.
7.	Create a multi-threaded Java application that simulates any real time application with required functionalities. For eg. Basic chat system in which each user (thread) sends and receives messages. Use isAlive() to check the status of threads and join() to ensure proper synchronization. Implement thread priorities to handle high-priority messages and demonstrate thread suspension, resumption, and stopping.
	Part II : Computer Graphics
SR.No.	**Group B (Any SIX)**
1.	Write a C/C++/Java program to draw the following pattern using (a) The DDA line drawing algorithm for both rectangles with Dotted, Thick line style and (b) Bresenham‘s line drawing algorithm for a diamond shape with Dashed, Solid line style. 

2.	Write a menu driven program in C/C++/Java to draw circle using DDA, Bresenham's , Midpoint circle drawing algorithm with different styles as solid, dotted and dashed circles.
3.	Write a menu driven program in C/C++/Java to draw a concave polygon a d fill it with the desired color using the scan fill algorithm; flood fill and seed fill algorithms.
4.	Write a program to implement the Sutherland-Hodgeman algorithm for clipping any polygon. Provide the vertices of the polygon to be clipped and the pattern of clipping interactively.
5.	Write a C/C++/Java program to implement the Cohen-Sutherland line clipping algorithm.
6.	Write a C/C++/Java program to implement translation, rotation, shear and scaling transformations on a 2D object about X axis, Y axis.
7.	Write C/C++/Java program to implement translation, sheer, rotation and scaling transformations on equilateral triangle and rhombus.
8.	Write a C/C++/Java program to implement rotation of a 2D object about X axis and an arbitrary point.

Sr. No.	Group C - Mini Project
Mini-Project using maximum features of Object-Oriented programming to develop solutions for any one application below:	
1.	Banking system having the following operations: a. Create an account b. Deposit money c. Withdraw money d. Honor daily withdrawal limit e. Check the balance f. Display Account information. g. Passbook Print (from to)
2.	Inventory management system having the following operations: a. List of all products b. Display individual product information c. Purchase d. Shipping e. Balance stock f. Loss and Profit calculation. g. Purchase Report (from to)
Note– Subject Incharge can consider any other application having similar features and complexity.	
	Learning Resources

1.	**Text Books:** 1. E Balaguruswamy, (2023). Programming with JAVA: A Primer. 7th edition. India: McGraw Hill Education. 2. Herbert Schildt, (2021). Java: The complete reference, 13th edition. McGraw-Hill Education. **Reference Books:** 1. Paul Deitel and Harvey Detail, Java: How to Program, Pearson's Publication, 9thEdition 2. Horstmann, C. S. (2023). Core Java - Vol. I – Fundamentals (Vol. 12). Pearson Education 3. Dr. Samit Bhattacharya, Computer Graphics, Oxford University Press, ISBN-13: 978-0-19-809619-1; ISBN-10: 0-19-809619-4. 4. D. Rogers, "Procedural Elements for Computer Graphics", 2ndEdition, Tata McGraw-Hill Publication, 2001, ISBN 0 – 07 – 047371 – 4. MOOC / NPTEL/YouTube Links: - 1. Programming In Java: https://onlinecourses.nptel.ac.in/noc25_cs57/preview 2. https://archive.nptel.ac.in/courses/106/103/106103224/ 3. https://archive.nptel.ac.in/courses/106/102/106102065/ 4. https://nptel.ac.in/courses/106106090 E-Books : 1. https://www.iitg.ac.in/samit/Computer%20Graphics.pdf 2. https://open.umn.edu/opentextbooks/textbooks/introduction-to-computer-graphics 3. http://www2.cs.uidaho.edu/~jeffery/courses/324/lecture.html

@The CO-PO mapping table

PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1		1	2	1								
CO2		1	2	1								
CO3	2	1	1									
CO4	1	2	2	1								
CO5		2	2	1								

Assignment No. 1

Title	Java Calculator Program with Dynamic Input Using Loops and Conditional Logic
Aim/Problem Statement	Implement a robust Java calculator program that captures user input dynamically, processes mathematical operations using conditional logic and looping constructs, and ensures efficient error handling.
CO Mapped	CO 1
Pre -requisite	· Basic Java programming · Control flow: if, switch, while, do-while, for · Scanner class for user input · Understanding of operators and type conversion · Java Development Kit (JDK) installed
Learning Objective	· To apply logical validation without exception handling · To develop interactive console applications in Java · To understand conditional branching and loops · To use mathematical operators effectively · To create user-friendly and reusable code blocks

Theory:

What is a Calculator Program?

A calculator simulates arithmetic operations in a software form, allowing users to input numbers and select operations.

User Input with Scanner Class

Scanner is used to read numbers and operation choices from the user.

```
Scanner sc = new Scanner(System.in);

int a = sc.nextInt();

char operator = sc.next().charAt(0);
```

Control Flow and Conditional Logic

The core logic of the calculator uses:

- switch or if-else statements to determine which operation to perform
- while or do-while loops to keep the program running until the user exits

Sample Operations Supported

- Addition (+)
- Subtraction (-)

- Multiplication (*)
- Division (/)
- Modulus (%)

Switch Statement

To select the operation:

```
switch (operator)
{
        case '+':
    result = a + b;
    break;
        ...
}
```

Conclusion:

The calculator project strengthens the foundational programming skills in Java by integrating conditional logic, looping structures, user interaction,

Questions:

1. What is the role of loops in this program?
2. How can we validate an invalid menu choice (e.g., wrong operator)?
3. How would you extend this calculator to support floating-point inputs?
4. How does this calculator prevent divide-by-zero without using exception handling?

Assignment No. 2

Title	Java E-commerce Order Processing System Using Constructor Overloading
Aim/Problem Statement	Develop a Java program for an E-commerce order processing where some products are initialized through multiple constructors, overloaded constructors, where users can input some product details manually, the system computes total order cost dynamically, applies discount policies based on conditions, and presents a detailed invoice summarizing the purchase.
CO Mapped	CO 2
Pre -requisite	• Core Java programming (classes, objects, arrays, control statements) • Constructor and constructor overloading • Scanner class for input • Basic understanding of conditional logic and loops
Learning Objective	• To learn constructor overloading and dynamic object creation • To apply conditional logic for discounts and validations • To develop an interactive real-world simulation in Java • To display structured output (invoice generation)

Theory:

What is Constructor Overloading?

Constructor overloading allows a class to have more than one constructor, each with different parameters. This supports flexible object creation based on the situation.

E-commerce System Workflow:

1. **Product Initialization:**
   - Some products are hard-coded using overloaded constructors.
   - Some products are taken as input from the user.
2. **Input Validation (Without Exceptions):**
   - Use conditional logic (`if`) to ensure correct inputs.
   - Skip or repeat inputs if invalid (e.g., negative quantity or price).
3. **Order Processing:**
   - Calculate cost = price × quantity
   - Apply discount if applicable
   - Calculate final payable amount
4. **Invoice Generation:**
   - List all items
   - Show totals, discount, and final payable amount

Sample Discount Logic:

- If total > ₹5000 → 10% discount

- If total > ₹10000 → 15% discount

Constructor syntax in Java, including examples of default, parameterized, and overloaded constructors:

Default Constructor

A constructor with no parameters.

```
class Product {
        String name;
        int price;
        // Default constructor
        Product() {
        name = "Unknown";
        price = 0;
        }
}
```

Parameterized Constructor

Used to initialize objects with specific values.

```
class Product {
        String name;
        int price;
 // Parameterized constructor
   Product(String n, int p) {
        name = n;
        price = p;
        }
}
```

Constructor Overloading

Multiple constructors in the same class with different parameter lists.

```
class Product {
        String name;
        int price;
        int quantity;
 // Constructor 1: only name
   Product(String n) {
        name = n;
        price = 100;    // default price
      quantity = 1;
        }
 // Constructor 2: name and price
   Product(String n, int p) {
        name = n;
        price = p;
      quantity = 1;
        }
 // Constructor 3: name, price, and quantity
   Product(String n, int p, int q) {
```

```
        name = n;
        price = p;
      quantity = q;
        }
}
```

Conclusion

This project highlights the use of OOP principles like class design and constructor overloading. It demonstrates how a complete order processing system can be developed using simple Java tools and logical validation instead of exception handling.

Questions

1. How does constructor overloading help in flexible product initialization?
2. How are discount rules applied dynamically?
3. What would happen if the user entered a negative price or quantity? How is it avoided?
4. How would you extend this system to support more complex pricing features like GST or shipping?
5. How does this system validate user inputs without using exception handling?

Assignment No. 5

Title	Java Multi-threaded Chat Simulation Project
Aim/Problem Statement	Create a multi-threaded Java application that simulates any real time application with required functionalities. For eg. Basic chat system in which each user(thread) sends and receives messages. Use isAlive() to check the status of threads and join() to ensure proper synchronization. Implement thread priorities to handle high-priority messages and Demonstrate thread suspension, resumption, and stopping.
CO Mapped	CO 3
Pre -requisite	1. Basic programming skills in Java 2. 64-bit Open Source Linux (e.g., Ubuntu) 3. Open-source Java Development Kit (JDK), Java compiler (javac), or IDE (e.g., Eclipse, IntelliJ, VS code)
Learning Objective	To learn and apply multi-threading concepts to develop concurrent applications.

Theory:

What is Multi-threading?

Multi-threading is a core feature of Java that allows concurrent execution of two or more parts of a program for maximum CPU utilization. Each part of such a program is called a thread, and each thread defines a separate path of execution.

Real-Time Simulation – Chat System

In a real-time chat system, multiple users send and receive messages simultaneously. Each user can be modeled as an independent thread. The messages can be shared using synchronized structures like queues or shared objects to avoid race conditions.

Thread Lifecycle

Java threads have the following states:

1. **New** – Thread object created.
2. **Runnable** – Thread is ready to run.
3. **Running** – Thread is executing.
4. **Blocked/Waiting** – Waiting for a monitor lock or other threads.
5. **Terminated** – Execution completed or stopped.

Key Java Thread Methods

- start(): Begins thread execution.
- run(): Contains code to be executed in the thread.

- join(): Waits for the thread to die.
- isAlive(): Checks if thread is still running.
- setPriority(): Sets priority (1–10).
- wait()/notify(): Used for thread communication via monitors.

Thread Control (Suspension/Resumption)

Due to the deprecation of suspend(), resume(), and stop(), we use custom boolean flags and synchronization to control thread execution safely.

```
volatile boolean suspended = false;
volatile boolean stopped = false;
```

Threads can **pause** execution using:

```
while (suspended)
{
   wait();
}
```

They can resume using:

```
suspended = false;
notify();
```

Thread Priority

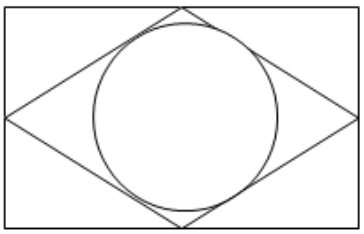
Java allows setting thread priorities between Thread.MIN_PRIORITY (1) and Thread.MAX_PRIORITY (10). Higher-priority threads are scheduled more frequently, useful for simulating critical user actions or messages.

Conclusion:

Questions:

1. How does the join() method help in synchronizing threads in the Java chat application? Explain the concept of operator overloading with examples.
2. Why are suspend() and resume() methods not used directly in Java for thread control, and how is their functionality simulated in the program?

Assignment No. 1

Title	Pattern drawing using line and circle.
Aim/Problem Statement	Write a C/C++/Java program to draw the following pattern using (c) The DDA line drawing algorithm for both rectangles with Dotted, Thick line style and (d) Bresenham's line drawing algorithm for a diamond shape with Dashed, Solid line style. 
CO Mapped	CO4
Pre -requisite	1. Basic programming skills of C++ 2. 64-bit Open source Linux 3. Open Source C++ Programming tool like G++/GCC
Learning Objective	To learn and apply DDA line and Bresenham's circle drawing algorithm.

Theory:

DDA Line Drawing Algorithm:

Line is a basic element in graphics. To draw a line, you need two end points between which you can draw a line. Digital Differential Analyzer (DDA) line drawing algorithm is the simplest line drawing algorithm in computer graphics. It works on an incremental method. It plots the points from starting point of line to end point of line by incrementing in X and Y direction in each iteration.
DDA line drawing algorithm works as follows:

Step 1: Get coordinates of both the end points (X_1, Y_1) and (X_2, Y_2) from the user.

Step 2: Calculate the difference between two end points in X and Y direction.

$dx = X_2 – X_1;$
$dy = Y_2 – Y_1;$

Step 3: Based on the calculated difference in step-2, you need to identify the number of steps to put pixel. If dx > dy, then you need more steps in x coordinate; otherwise in y coordinate.

```
if (absolute(dx) > absolute(dy))
Steps = absolute(dx);
else
Steps = absolute(dy);
```

Step 4: Calculate the increment in x coordinate and y coordinate.

```
Xincrement = dx / (float) steps;
Yincrement = dy / (float) steps;
```

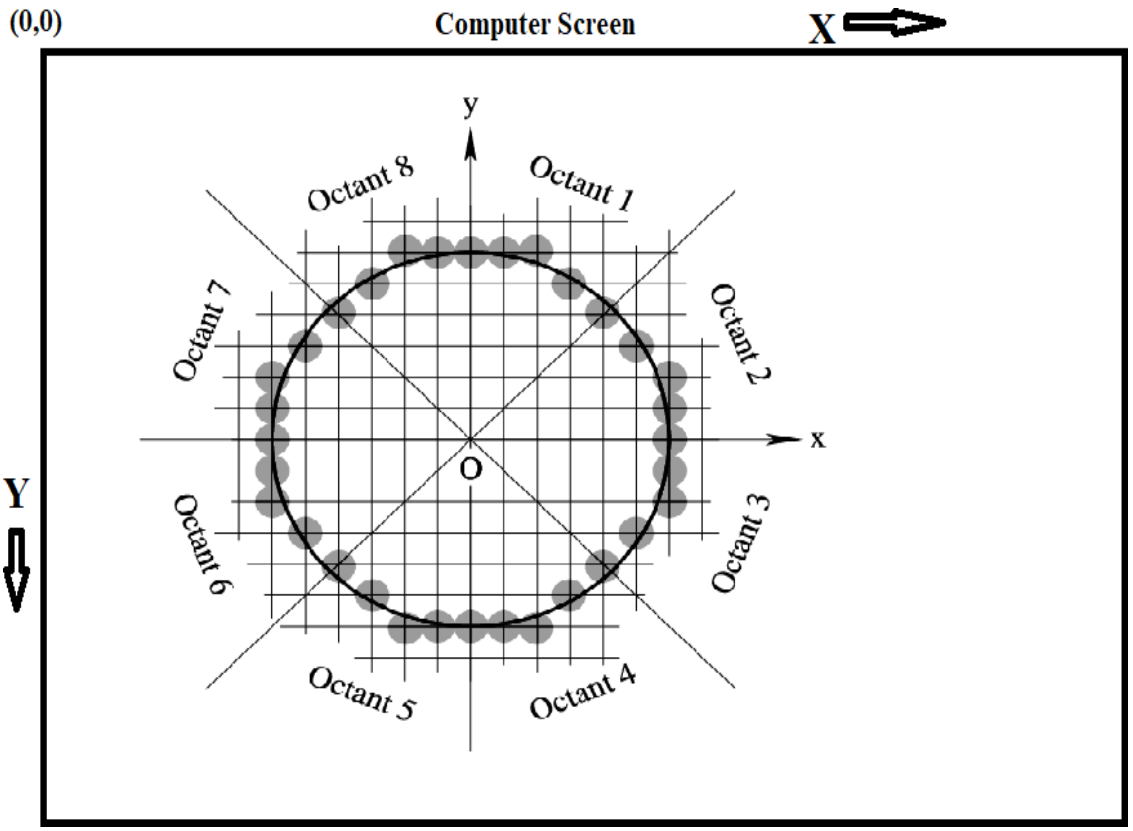
Step 5: Plot the pixels by successfully incrementing x and y coordinates accordingly and complete

the drawing of the line.

```
for(int i=0; i < Steps; i++)
{
                    X1 = X1 + Xincrement;
                    Y1 = Y1 + Yincrement;
                    putpixel(Round(X1), Round(Y1), ColorName);
}
```

2) Bresenham's Circle Drawing Algorithm:

A circle is an eight-way symmetric figure. The shape of the circle is the same in all quadrants. In each quadrant, there are two octants. If the calculation of the point of one octant is done, then the other seven points can be calculated easily by using the concept of eight-way symmetry.
Bresenham's Circle Drawing Algorithm is a circle drawing algorithm that selects the nearest pixel position to complete the arc. The unique part of this algorithm is that it uses only integer arithmetic which makes it significantly faster than other algorithms using floating point arithmetic.



Step 1: Read the x and y coordinates of center: (centx, centy)
Step 2: Read the radius of circle: (r)
Step 3: Initialize, x = 0; y = r;
Step 4: Initialize decision parameter: p = 3 – (2 * r)
Step 5:

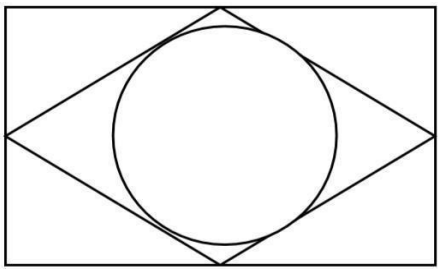
```
        do {
putpixel(centx+x, centy-y, ColorName);
if (p<0)
p = p+(4*x)+6;
else
{
p=p+[4*(x-y)]+10;
```

```
y=y-1;
}
x=x+1;
} while(x<y)
```

Here in step 5, putpixel() function is used which will print Octant-1 of the circle with radius r after the completion of all the iterations of the do-while loop. Because of the eight-way symmetry property of circle, we can draw other octants of the circle using following putpixel() function:

Octant 1: putpixel(centx+x, centy-y, ColorName);
Octant 2: putpixel(centx+y, centy-x, ColorName);
Octant 3: putpixel(centx+y, centy+x, ColorName);
Octant 4: putpixel(centx+x, centy+y, ColorName);
Octant 5: putpixel(centx-x, centy+y, ColorName);
Octant 6: putpixel(centx-y, centy+x, ColorName);
Octant 7: putpixel(centx-y, centy-x, ColorName);
Octant 8: putpixel(centx-x, centy-y, ColorName);

Drawing Pattern using lines and circle s:



To draw this pattern, we require two rectangles and one circle. We can use suitable geometry to get coordinates to draw rectangles and circle. Then we can call DDA line drawing and Bresenham's circle drawing algorithms by passing appropriate parameters to get the required pattern.

Conclusion:

Questions:

1. Explain the derivation of decision parameters in Bresenham's circle drawing algorithm.
2. Explain eight-way symmetric figure for circle drawing.
3. Explain the DDA line drawing algorithm.
4. What is step in DDA line drawing algorithm?
5. Explain various line styles used in computer graphics.

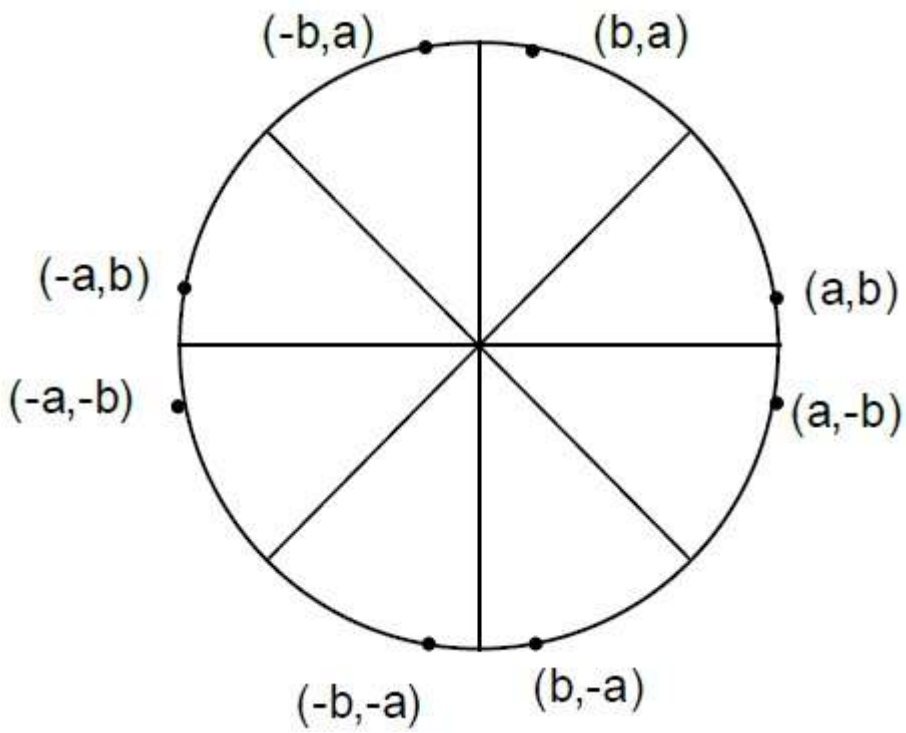
Assignment No. 2

Title	Circle drawing Algorithms.
Aim/Problem Statement	Write a menu driven program in C/C++/Java to draw circle using DDA, Bresenham's , Midpoint circle drawing algorithm with different styles as solid, dotted and dashed circles.
CO Mapped	CO 4
Pre -requisite	1. Basic programming skills of C++ 2. 64-bit Open source Linux 3. Open Source C++ Programming tool like G++/GCC
Learning Objective	To learn and apply DDA circle drawing, Bresenham's circle drawing and Midpoint circle drawing algorithm.

Theory:

Circle drawing algorithms are used in computer graphics to generate points on a raster (pixel grid) that approximate a circle. The goal is to calculate which pixels best represent a circle with a given center and radius.

The equation of circle is $X^2 + Y^2 = r^2$, where r is radius.



1) DDA circle drawing Algorithm:

The Digital Differential Analyzer (DDA) algorithm is a line drawing algorithm used in computer graphics to generate a line segment on a digital display. While primarily designed for lines, it can be adapted for circle drawing as well. The algorithm works by calculating the incremental differences between x and y coordinates to plot points along the line or curve.

- **Circle Equation (Parametric Form)**

The circle with center (xc, yc) and radius r can be defined parametrically as:

$$x = xc + r * \cos(\theta)$$
$$y = yc + r * \sin(\theta)$$

You step through values of θ from 0 to 2π, incrementing by a small delta (like 0.01 or smaller), and compute points.

- **Steps:**

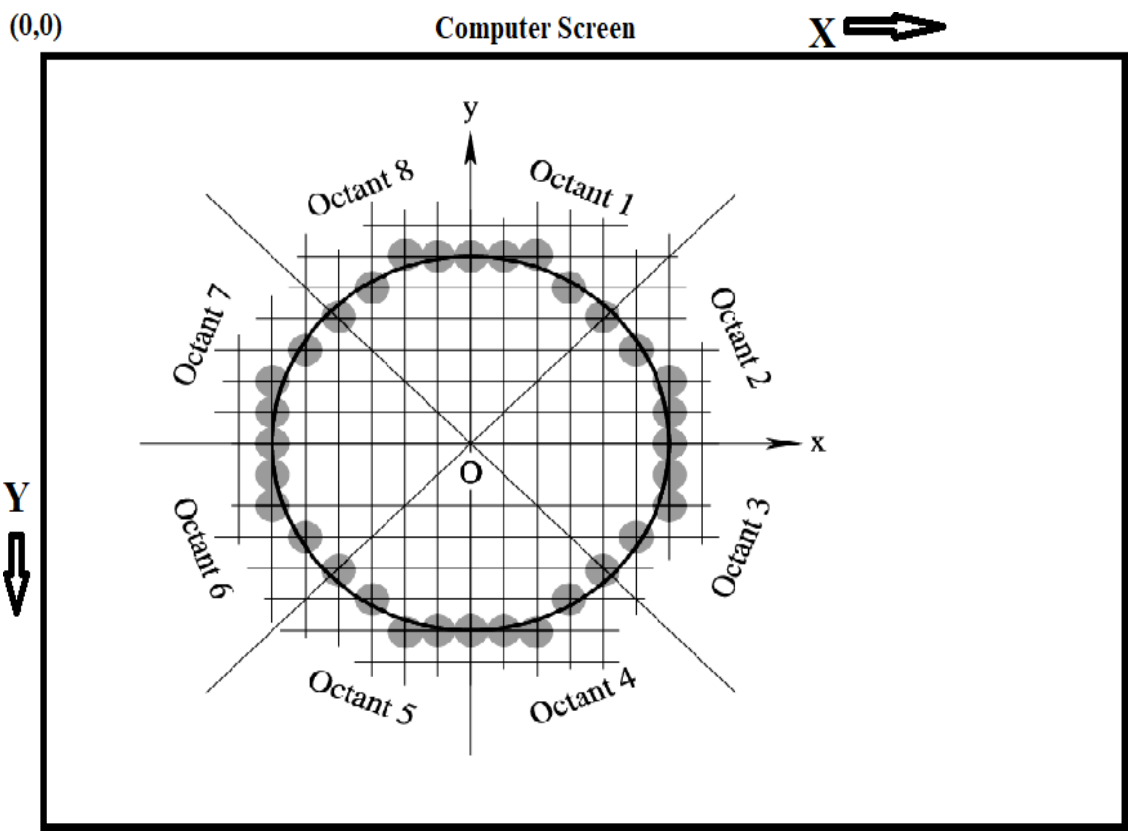
1. Choose a step size dθ (e.g., 0.01 radians).
2. For θ from 0 to 2π, compute:
   - x=xc+r·cos
   - y=yc+r·sin
3. Plot the pixel at (round(x),round(y))

Limitations of DDA Circle drawing algorithm:

- Involves floating-point and trigonometric calculations (cos, sin), making it **slower** than integer-only methods like **Midpoint or Bresenham's circle** algorithm.
- Not ideal for performance-critical graphics (like real-time rendering).

2) Bresenham's circle drawing Algorithm:

A circle is an eight-way symmetric figure. The shape of the circle is the same in all quadrants. In each quadrant, there are two octants. If the calculation of the point of one octant is done, then the other seven points can be calculated easily by using the concept of eight-way symmetry.
Bresenham's Circle Drawing Algorithm is a circle drawing algorithm that selects the nearest pixel position to complete the arc. The unique part of this algorithm is that it uses only integer arithmetic which makes it significantly faster than other algorithms using floating point arithmetic.



Step 1: Read the x and y coordinates of center: (centx, centy)

Step 2: Read the radius of circle: (r)
Step 3: Initialize, x = 0; y = r;
Step 4: Initialize decision parameter: p = 3 – (2 * r)
Step 5:

```
        do {
putpixel(centx+x, centy-y, ColorName);
if (p<0)
p = p+(4*x)+6;
else
{
p=p+[4*(x-y)]+10;
y=y-1;
}
x=x+1;
} while(x<y)
```

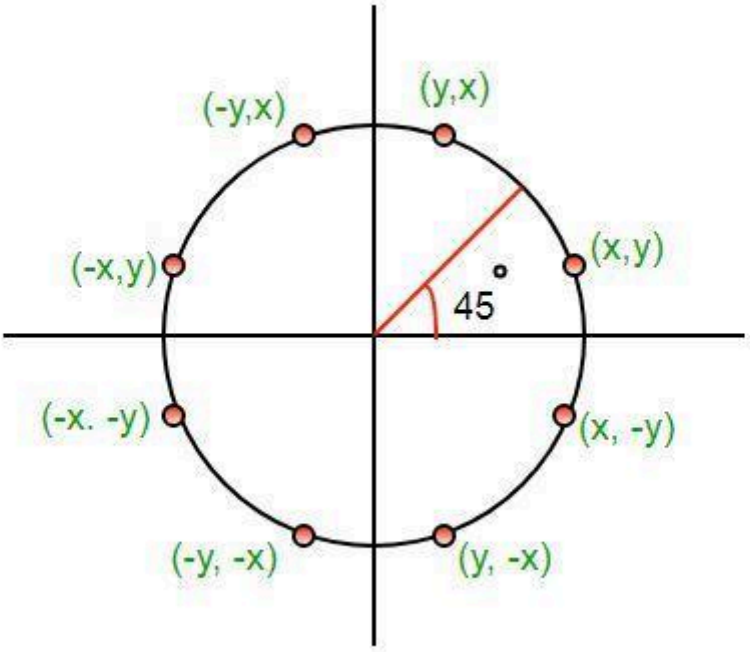
Here in step 5, putpixel() function is used which will print Octant-1 of the circle with radius r after the completion of all the iterations of the do-while loop. Because of the eight-way symmetry property of circle, we can draw other octants of the circle using following putpixel() functions:

Octant 1: putpixel(centx+x, centy-y, ColorName);
Octant 2: putpixel(centx+y, centy-x, ColorName);
Octant 3: putpixel(centx+y, centy+x, ColorName);
Octant 4: putpixel(centx+x, centy+y, ColorName);
Octant 5: putpixel(centx-x, centy+y, ColorName);
Octant 6: putpixel(centx-y, centy+x, ColorName);
Octant 7: putpixel(centx-y, centy-x, ColorName);
Octant 8: putpixel(centx-x, centy-y, ColorName);

3) Midpoint circle drawing Algorithm:

The **mid-point** circle drawing algorithm is an algorithm used to determine the points needed for rasterizing a circle.

We use the **mid-point** algorithm to calculate all the perimeter points of the circle in the **first octant** and then print them along with their mirror points in the other octants. This will work because a circle is symmetric about its centre.



Algorithm:
Given-

- Centre point of Circle = (X0, Y0)
- Radius of Circle = R

The points generation using Mid Point Circle Drawing Algorithm involves the following steps-

Step-01: Assign the starting point coordinates (X0, Y0) as-

$$X0 = 0$$
$$Y0 = R$$

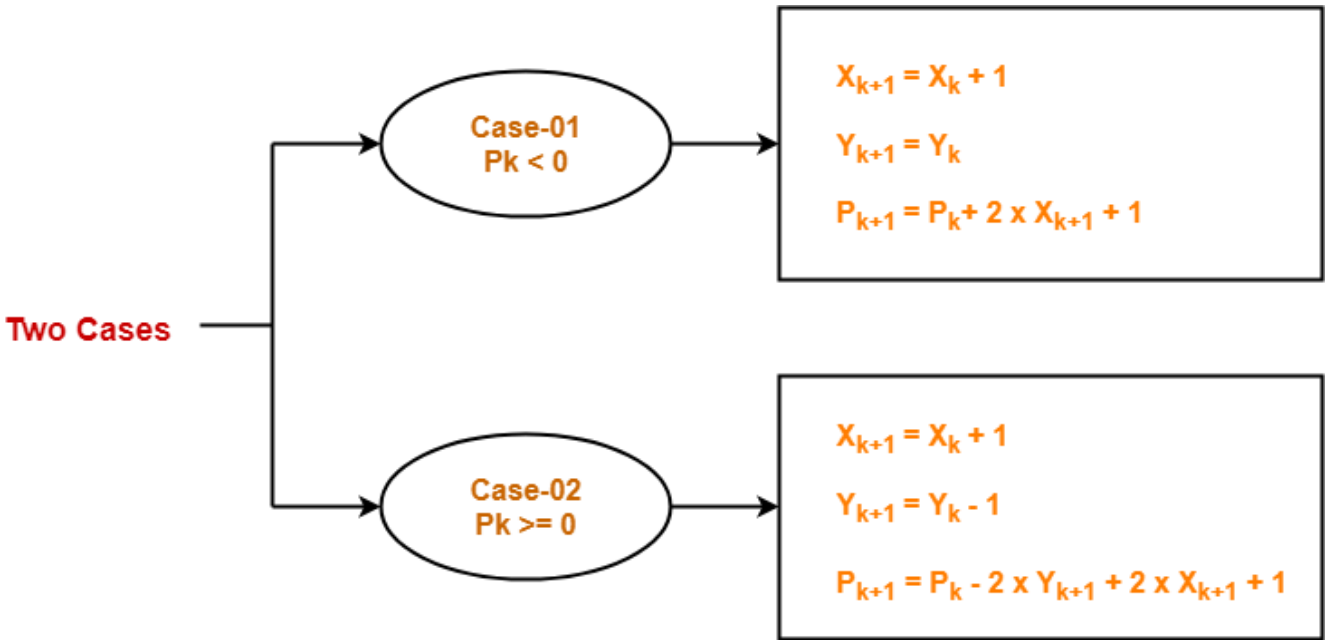
Step-02: Calculate the value of initial decision parameter P0 as-

$$P0 = 1 – R$$

Step-03: Suppose the current point is (Xk, Yk) and the next point is (Xk+1, Yk+1).

Find the next point of the first octant depending on the value of decision parameter Pk.

Follow the below two cases-



Step-04: If the given centre point (X0, Y0) is not (0, 0), then do the following and plot the point-

$$Xplot = Xc + X0$$
$$Yplot = Yc + Y0$$

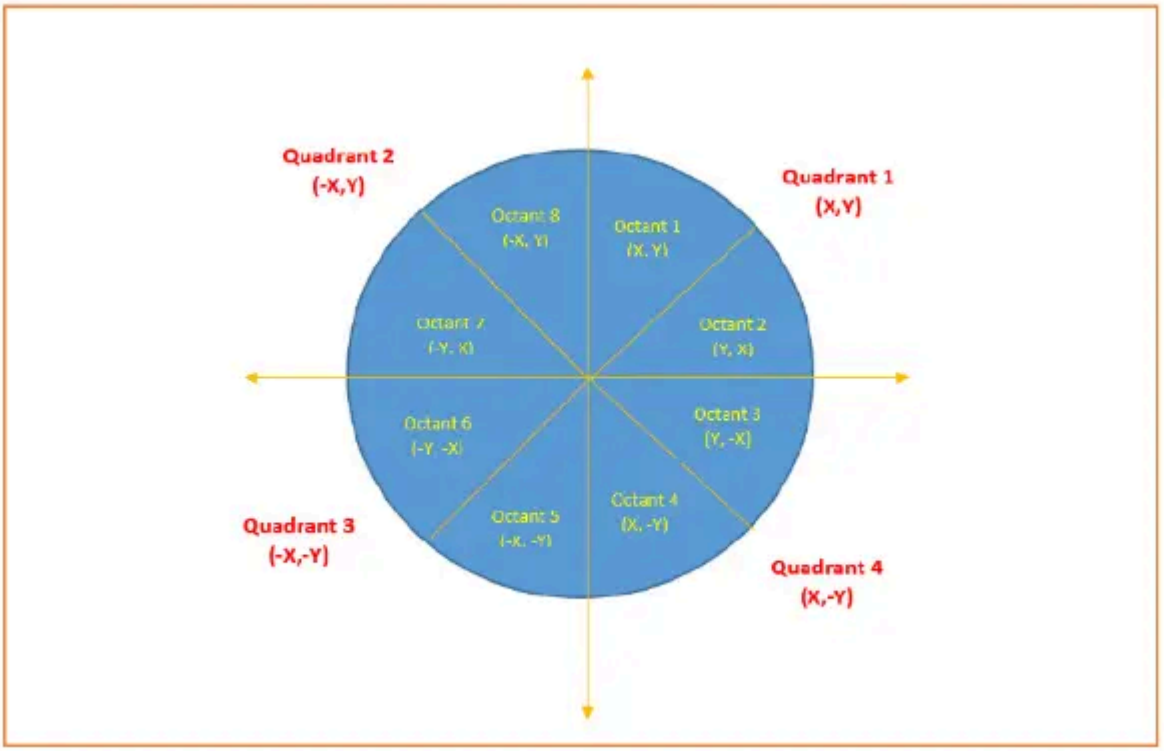
Here, (Xc, Yc) denotes the current value of X and Y coordinates.

Step-05: Keep repeating Step-03 and Step-04 until Xplot >= Yplot.

Step-06: Step-05 generates all the points for one octant.

To find the points for other seven octants, follow the eight symmetry property of circle.

This is depicted by the following figure-



Conclusion:

Questions:

1. Explain decision parameter.
2. Explain 8 way symmetry of circle.
3. Compare DDA,Midpoint and Bresenham's circle drawing algorithms.
4. What is the initial value of the decision parameter in all these algorithm?
5. Why do we only calculate points for one-eighth of the circle?
6. What are the limitations of the Midpoint Circle Algorithm?
7. How is the decision variable updated in Bresenham's Algorithm?